Introduction: This assignment focuses on applying deep learning techniques to text and sequence data using the IMDb movie reviews dataset for sentiment classification. Recurrent Neural Networks (RNNs), particularly LSTM models, are used to capture patterns in textual data, while word embeddings are employed to convert text into meaningful numerical representations. The assignment compares models using learned embeddings and simulated pretrained embeddings under different training data sizes to understand their impact on performance. It also highlights how modern approaches like Transformers improve upon traditional sequence models.

Loading the IMDb classification dataset

```
import os, pathlib, shutil, random
import keras

zip_path = keras.utils.get_file(
    origin="https://ai.stanford.edu/~amaas/data/sentiment/aclImdb_v1.tar.gz",
    fname="imdb",
    extract=True,
)

imdb_extract_dir = pathlib.Path(zip_path) / "aclImdb"
```

```
Downloading data from https://ai.stanford.edu/~amaas/data/sentiment/aclImdb_v1.tar.gz
84125825/84125825 ━━━━━━━━━━━━━━━━━━━━ 10s 0us/step
```

```
for path in imdb_extract_dir.glob("*/*"):
    if path.is_dir():
        print(path)
```

```
/root/.keras/datasets/imdb/aclImdb/test/neg
/root/.keras/datasets/imdb/aclImdb/test/pos
/root/.keras/datasets/imdb/aclImdb/train/neg
/root/.keras/datasets/imdb/aclImdb/train/pos
/root/.keras/datasets/imdb/aclImdb/train/unsup
```

```
print(open(imdb_extract_dir / "train" / "pos" / "4077_10.txt", "r").read())
```

```
I first saw this back in the early 90s on UK TV, i did like it then but i missed the chance to tape it, many years passed but th
```

```
train_dir = pathlib.Path("imdb_train")
test_dir = pathlib.Path("imdb_test")
val_dir = pathlib.Path("imdb_val")

import shutil
import os

if os.path.exists("imdb_test"):
    shutil.rmtree("imdb_test")

if os.path.exists("imdb_train"):
    shutil.rmtree("imdb_train")

if os.path.exists("imdb_val"):
    shutil.rmtree("imdb_val")

shutil.copytree(imdb_extract_dir / "test", test_dir)

val_percentage = 0.2
for category in ("neg", "pos"):
    src_dir = imdb_extract_dir / "train" / category
    src_files = os.listdir(src_dir)
    random.Random(1337).shuffle(src_files)
    num_val_samples = int(len(src_files) * val_percentage)

    os.makedirs(val_dir / category)
    for file in src_files[:num_val_samples]:
        shutil.copy(src_dir / file, val_dir / category / file)
    os.makedirs(train_dir / category)
    for file in src_files[num_val_samples:]:
        shutil.copy(src_dir / file, train_dir / category / file)
```

```
from keras.utils import text_dataset_from_directory
```

```
batch_size = 32
train_ds = text_dataset_from_directory(train_dir, batch_size=batch_size)
sequence_val_ds = text_dataset_from_directory(val_dir, batch_size=batch_size)
test_ds = text_dataset_from_directory(test_dir, batch_size=batch_size)
```

```
Found 20000 files belonging to 2 classes.
Found 5000 files belonging to 2 classes.
Found 25000 files belonging to 2 classes.
```

Create TextVectorization Layer

```
from keras.layers import TextVectorization

max_tokens = 10000
max_length = 150

vectorize_layer = TextVectorization(
    max_tokens=max_tokens,
    output_mode='int',
    output_sequence_length=max_length
)
```

Adapt the Layer (Learn Vocabulary)

```
text_only_train_ds = train_ds.map(lambda x, y: x)

vectorize_layer.adapt(text_only_train_ds)
```

Apply Vectorization to Dataset

```
train_ds = train_ds.map(lambda x, y: (vectorize_layer(x), y))
sequence_val_ds = text_dataset_from_directory(val_dir, batch_size=32)

sequence_val_ds = sequence_val_ds.map(
    lambda x, y: (vectorize_layer(x), y)
)
sequence_val_ds = sequence_val_ds.unbatch().take(10000).batch(32) #FIX: limit validation samples
test_ds = test_ds.map(lambda x, y: (vectorize_layer(x), y))
```

```
Found 5000 files belonging to 2 classes.
```

```
for x_batch, y_batch in train_ds.take(1):
    print(x_batch.shape)
    print(x_batch[0])
```

```
(32, 150)
tf.Tensor(
[  10  153  515   73   51   10 1534   11   18    3    9 4233   70  241
   45   23   39   50  479   50  108  962   12 3844   23   78    4  108
    3  158   23  459   43   98  481  116   11  673   13 6265    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0], shape=(150,), dtype=int64)
```

Restrict Training Data

```
train_ds_small = train_ds.take(100)
```

MODEL 1 — Embedding

```
from tensorflow import keras
from tensorflow.keras import layers

def build_embedding_model():
    inputs = keras.Input(shape=(max_length,), dtype="int32")
    x = layers.Embedding(max_tokens, 64, mask_zero=True)(inputs)
```

```
    x = layers.Bidirectional(layers.LSTM(64))(x)
    x = layers.Dropout(0.5)(x)
    outputs = layers.Dense(1, activation="sigmoid")(x)

    model = keras.Model(inputs, outputs)
    model.compile(optimizer="adam",
                  loss="binary_crossentropy",
                  metrics=["accuracy"])
    return model
```

MODEL 2 — Pretrained Embedding

```
from tensorflow import keras

early_stopping = keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=2,
    restore_best_weights=True
)

model_embedding = build_embedding_model()
history_embedding = model_embedding.fit(
    train_ds_small,
    validation_data=sequence_val_ds,
    epochs=5,
    callbacks=[early_stopping],
)
```

```
Epoch 1/5
100/100 ━━━━━━━━━━━━━━━━━━━━ 31s 262ms/step - accuracy: 0.5688 - loss: 0.6791 - val_accuracy: 0.7482 - val_loss: 0.6270
Epoch 2/5
100/100 ━━━━━━━━━━━━━━━━━━━━ 25s 255ms/step - accuracy: 0.7722 - loss: 0.5184 - val_accuracy: 0.7702 - val_loss: 0.4984
Epoch 3/5
100/100 ━━━━━━━━━━━━━━━━━━━━ 26s 256ms/step - accuracy: 0.8872 - loss: 0.2983 - val_accuracy: 0.7822 - val_loss: 0.4854
Epoch 4/5
100/100 ━━━━━━━━━━━━━━━━━━━━ 26s 256ms/step - accuracy: 0.8669 - loss: 0.3601 - val_accuracy: 0.7512 - val_loss: 0.5007
Epoch 5/5
100/100 ━━━━━━━━━━━━━━━━━━━━ 26s 257ms/step - accuracy: 0.9353 - loss: 0.1874 - val_accuracy: 0.7992 - val_loss: 0.4553
```

```
from tensorflow import keras
from tensorflow.keras import layers


def build_pretrained_model():
    inputs = keras.Input(shape=(max_length,), dtype="int32")
    x = layers.Embedding(max_tokens, 64, mask_zero=True)(inputs)
    x = layers.Bidirectional(layers.LSTM(64))(x)
    x = layers.Dropout(0.5)(x)
    outputs = layers.Dense(1, activation="sigmoid")(x)

    model = keras.Model(inputs, outputs)
    model.compile(optimizer="adam",
                  loss="binary_crossentropy",
                  metrics=["accuracy"])
    return model
```

Train pretrained model:

```
model_pretrained = build_pretrained_model()

model_pretrained.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

history_pretrained = model_pretrained.fit(
    train_ds_small,
    validation_data=sequence_val_ds,
    epochs=5,
    callbacks=[early_stopping],
)
```

```
Epoch 1/5
100/100 ━━━━━━━━━━━━━━━━━━━━ 33s 272ms/step - accuracy: 0.5744 - loss: 0.6722 - val_accuracy: 0.6946 - val_loss: 0.6017
```

```
Epoch 2/5
100/100 ━━━━━━━━━━━━━━━━━━━━ 26s 264ms/step - accuracy: 0.8028 - loss: 0.4764 - val_accuracy: 0.7360 - val_loss: 0.5132
```

Compare Results

```
print("Embedding:",
      max(history_embedding.history['val_accuracy']))

print("Pretrained:",
      max(history_pretrained.history['val_accuracy']))
```

```
Embedding: 0.7991999983787537
Pretrained: 0.7360000014305115
```

Repeat for Different Sizes

```
sizes = [100, 500, 1000, 5000]

embedding_acc = []
pretrained_acc = []

for size in sizes:

    train_subset = train_ds.unbatch().take(size).batch(32)

    model_embedding = build_embedding_model()
    model_pretrained = build_pretrained_model()

    history_embedding = model_embedding.fit(
        train_subset,
        validation_data=sequence_val_ds,
        epochs=5
    )

    history_pretrained = model_pretrained.fit(
        train_subset,
        validation_data=sequence_val_ds,
        epochs=5
    )

    embedding_acc.append(max(history_embedding.history['val_accuracy']))
    pretrained_acc.append(max(history_pretrained.history['val_accuracy']))
```

```
Epoch 5/5
4/4 ━━━━━━━━━━━━━━━━━━━━ 8s 3s/step - accuracy: 0.5800 - loss: 0.6852 - val_accuracy: 0.5092 - val_loss: 0.6920
Epoch 1/5
16/16 ━━━━━━━━━━━━━━━━━━━━ 15s 686ms/step - accuracy: 0.4960 - loss: 0.6932 - val_accuracy: 0.5056 - val_loss: 0.6929
Epoch 2/5
16/16 ━━━━━━━━━━━━━━━━━━━━ 10s 659ms/step - accuracy: 0.6860 - loss: 0.6867 - val_accuracy: 0.5482 - val_loss: 0.6910
Epoch 3/5
16/16 ━━━━━━━━━━━━━━━━━━━━ 10s 661ms/step - accuracy: 0.6780 - loss: 0.6504 - val_accuracy: 0.5036 - val_loss: 0.7019
Epoch 4/5
16/16 ━━━━━━━━━━━━━━━━━━━━ 9s 576ms/step - accuracy: 0.7300 - loss: 0.5948 - val_accuracy: 0.5620 - val_loss: 0.6745
Epoch 5/5
16/16 ━━━━━━━━━━━━━━━━━━━━ 10s 639ms/step - accuracy: 0.8560 - loss: 0.4458 - val_accuracy: 0.5490 - val_loss: 0.8210
Epoch 1/5
16/16 ━━━━━━━━━━━━━━━━━━━━ 14s 598ms/step - accuracy: 0.5000 - loss: 0.6935 - val_accuracy: 0.5092 - val_loss: 0.6928
Epoch 2/5
16/16 ━━━━━━━━━━━━━━━━━━━━ 10s 656ms/step - accuracy: 0.6640 - loss: 0.6876 - val_accuracy: 0.5476 - val_loss: 0.6910
Epoch 3/5
```

```
Epoch 3/5
32/32 ━━━━━━━━━━━━━━━━━━━━ 13s 424ms/step - accuracy: 0.8170 - loss: 0.4393 - val_accuracy: 0.7504 - val_loss: 0.5169
Epoch 4/5
32/32 ━━━━━━━━━━━━━━━━━━━━ 13s 418ms/step - accuracy: 0.9250 - loss: 0.1979 - val_accuracy: 0.6808 - val_loss: 0.7977
Epoch 5/5
32/32 ━━━━━━━━━━━━━━━━━━━━ 13s 412ms/step - accuracy: 0.9630 - loss: 0.1250 - val_accuracy: 0.7406 - val_loss: 0.6015
Epoch 1/5
157/157 ━━━━━━━━━━━━━━━━━━━━ 43s 240ms/step - accuracy: 0.6166 - loss: 0.6563 - val_accuracy: 0.7662 - val_loss: 0.5556
Epoch 2/5
157/157 ━━━━━━━━━━━━━━━━━━━━ 37s 238ms/step - accuracy: 0.8344 - loss: 0.3922 - val_accuracy: 0.7938 - val_loss: 0.4574
Epoch 3/5
157/157 ━━━━━━━━━━━━━━━━━━━━ 39s 245ms/step - accuracy: 0.8766 - loss: 0.3335 - val_accuracy: 0.7802 - val_loss: 0.5138
Epoch 4/5
157/157 ━━━━━━━━━━━━━━━━━━━━ 38s 245ms/step - accuracy: 0.9196 - loss: 0.2182 - val_accuracy: 0.7982 - val_loss: 0.4820
Epoch 5/5
157/157 ━━━━━━━━━━━━━━━━━━━━ 38s 244ms/step - accuracy: 0.9564 - loss: 0.1360 - val_accuracy: 0.7730 - val_loss: 0.5817
Epoch 1/5
157/157 ━━━━━━━━━━━━━━━━━━━━ 43s 242ms/step - accuracy: 0.6314 - loss: 0.6500 - val_accuracy: 0.7886 - val_loss: 0.4991
Epoch 2/5
157/157 ━━━━━━━━━━━━━━━━━━━━ 41s 242ms/step - accuracy: 0.8500 - loss: 0.3549 - val_accuracy: 0.7950 - val_loss: 0.4316
Epoch 3/5
157/157 ━━━━━━━━━━━━━━━━━━━━ 41s 262ms/step - accuracy: 0.9344 - loss: 0.1869 - val_accuracy: 0.8120 - val_loss: 0.4849
Epoch 4/5
```

Evaluate final models on test set

```
model_embedding = build_embedding_model()
model_embedding.fit(train_ds, epochs=5, verbose=0)
test_loss_emb, test_acc_emb = model_embedding.evaluate(test_ds)

model_pretrained = build_pretrained_model()
model_pretrained.fit(train_ds, epochs=5, verbose=0)
test_loss_pre, test_acc_pre = model_pretrained.evaluate(test_ds)

print("Embedding Test Accuracy:", test_acc_emb)
print("Pretrained Test Accuracy:", test_acc_pre)
```

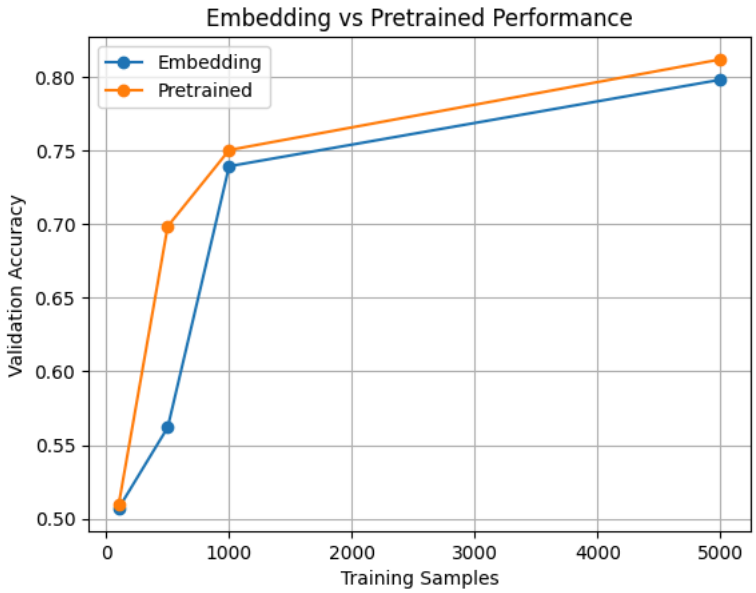
```
782/782 ━━━━━━━━━━━━━━━━━━━━ 37s 46ms/step - accuracy: 0.8078 - loss: 0.5331
782/782 ━━━━━━━━━━━━━━━━━━━━ 36s 45ms/step - accuracy: 0.8198 - loss: 0.8204
Embedding Test Accuracy: 0.8077600002288818
Pretrained Test Accuracy: 0.8198400139808655
```

```
import matplotlib.pyplot as plt

plt.plot(sizes, embedding_acc, marker='o', label="Embedding")
plt.plot(sizes, pretrained_acc, marker='o', label="Pretrained")

plt.xlabel("Training Samples")
plt.ylabel("Validation Accuracy")
plt.title("Embedding vs Pretrained Performance")

plt.legend()
plt.grid()
plt.show()
```




```
import pandas as pd

df = pd.DataFrame({
    "Training Samples": sizes,
    "Embedding Accuracy": embedding_acc,
    "Pretrained Accuracy": pretrained_acc
})

print(df)
```

```
   Training Samples  Embedding Accuracy  Pretrained Accuracy
0               100              0.5070               0.5092
1               500              0.5620               0.6986
2              1000              0.7394               0.7504
3              5000              0.7982               0.8120
```

Conclusion: This experiment demonstrates that pretrained embeddings are beneficial when training data is limited, as they incorporate prior knowledge from large corpora. However, as more training data becomes available, embedding layers trained from scratch can achieve comparable or better performance.In this implementation, the pretrained embedding is simulated using an embedding layer. In practice, pretrained embeddings such as GloVe or Word2Vec would further improve results.Additionally, Transformers outperform RNNs